A CASE-STUDY BASED REPORT

ON

Insider threat detection System

submitted in partial fulfilment of the requirements for the Course

MACHINE LEARNING

BACHELOR OF TECHNOLOGY

in

A.Y.: 2026-27

by

Niyathi Konduparthy **2520090072**

Rasagnya Machanavajhala **2520090018**

Chakrika Reddy **2520090044**

Under the Esteemed guidance of

Dr.Y.Lakshmi Prasanna

Asst.Professor, Dept. of CS&IT



(DEEMED TO BE UNIVERSITY)

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

INDEX

Abstract

Introduction.....

Background .....
Motivation for the Project .....
Overview of the Work Carried Out.....
Scope and Responsible-Use Statement.....

Problem Statement.....

Case-Study Scenario .....
Central Case-Study Question .....
Formal Problem Definition .....
Objectives that Guide the Study.....
Assumptions and Limitations.....

Dataset and Exploratory Data Analysis

About the Dataset .....
Purpose of EDA .....
Preparation Steps Actually Performed.....
EDA Exploration - Implementation.....
Summary of EDA Findings.....

Algorithm Selection and Implementation.....

Conclusion

Signature of the Team members

Abstract:

Insider threats pose a significant cybersecurity risk, as they originate from authorized users who already possess legitimate access to an organization's systems and sensitive data. This project proposes a Machine Learning-based Insider Threat Detection System that analyzes user activity data — including login patterns, failed login attempts, after-hours access, file and USB usage, downloads, and cloud uploads — to identify behavioral anomalies indicative of potential insider threats.

The collected data is preprocessed through cleaning, feature selection, encoding, and normalization to ensure reliable model performance. For classification, the system employs Elastic Net Regularized Logistic Regression, which combines L1 and L2 penalties to select relevant behavioral features, reduce overfitting, and handle correlated variables effectively. The model is trained and evaluated using precision, recall, F1-score, and AUC-ROC to ensure reliable detection while minimizing false positives.

Based on the analyzed behavior, users are assigned a risk score reflecting their likelihood of insider threat involvement. An administrative dashboard visualizes user activity, risk levels, and the key behavioral factors driving each classification, giving security teams interpretable insight into flagged cases. By enabling early identification of anomalous activity, the proposed system supports proactive investigation and helps organizations mitigate insider threats before they result in data loss or security breaches.

1. Introduction:

Background

In today's digital world, organizations rely heavily on computer networks, databases, and information systems to store and manage sensitive information. Although cybersecurity measures are designed to protect these systems from external attacks, threats can also originate from within an organization. Such threats, known as insider threats, may arise from employees or authorized users who intentionally or unintentionally misuse their access privileges. Traditional security systems often struggle to identify these threats because legitimate users may exhibit suspicious behavior while still having authorized access to organizational resources.

Machine learning provides an effective approach to analyzing user activity patterns and identifying potentially abnormal behavior. This project, titled **"Insider Threat Detection Using Elastic Net Regularized Logistic Regression,"** aims to develop a machine learning-based system that analyzes user activity logs and classifies activities as normal or potentially threatening. The system uses behavioral indicators such as login patterns, USB device usage, file access, email activity, web browsing, and after-hours access. Elastic Net regularization combines L1 and L2 penalties to support feature selection, manage correlated features, and reduce overfitting, helping build a more reliable predictive model.

Motivation for the Project

The increasing dependence on digital information systems has made the protection of confidential organizational data a major cybersecurity concern. Insider threats are particularly challenging to detect because authorized users may have legitimate access to sensitive resources, making suspicious activities difficult to distinguish from normal work behavior. Conventional rule-based security mechanisms may fail to recognize complex behavioral patterns or subtle deviations from established activity.

The motivation behind this project is to explore how machine learning can improve the early identification of potentially risky user behavior by analyzing activity logs and learning patterns from historical data. By implementing Elastic Net Regularized Logistic Regression, the project seeks to balance predictive performance with effective feature selection and model generalization. It also aims to present predictions and relevant risk indicators through a clear dashboard, supporting security teams in reviewing suspicious activities and making informed decisions while reducing unnecessary alerts.

Overview of the Work Carried Out

The project involves the design and development of a machine learning-based insider threat detection system using Elastic Net Regularized Logistic Regression. The proposed workflow begins with collecting or loading a suitable user activity dataset, followed by data preprocessing to handle missing values, inconsistent records, and unsuitable features. Feature engineering is then performed to derive meaningful behavioral indicators, including unusual login times, repeated login failures, removable-device usage, file downloads, sensitive-file access, email activity, and other potentially suspicious patterns.

After preprocessing, the relevant features are prepared for model training. An Elastic Net Regularized Logistic Regression model is developed to classify user activity into two categories: normal activity and potentially threatening activity. The model combines L1 and L2 regularization to help select useful features, manage correlated inputs, and control overfitting. Its performance can be evaluated using suitable classification metrics, including accuracy, precision, recall, F1-score, and ROC-AUC, where applicable.

The system also incorporates a dashboard concept for presenting user activity patterns, predicted risk levels, important contributing features, and potential threat alerts. This interface is intended to make model outputs easier to understand and review. The overall workflow consists of dataset management, preprocessing, feature engineering, model training and evaluation, risk prediction, and visualization of results. The system is designed as a decision-support tool for cybersecurity monitoring rather than an autonomous mechanism for determining employee misconduct.

Scope and Responsible-Use Statement

The scope of this project is limited to the development and evaluation of a machine learning-based approach for identifying potentially suspicious user behavior from available activity logs. The system focuses on binary classification using Elastic Net Regularized Logistic Regression and behavioral indicators such as login activity, device usage, file access, email patterns, and web activity. It also explores the presentation of predictions, risk indicators, and relevant behavioral features through a dashboard to support cybersecurity analysis.

The system's performance depends on the quality, completeness, representativeness, and labeling of the dataset used for training and evaluation. Therefore, predictions should be interpreted as indicators for further investigation rather than definitive evidence of malicious intent. The model may produce false positives and false negatives, and its outputs should be validated before any consequential decisions are made.

Responsible use is a fundamental consideration of this project. User activity data should be collected and processed only with appropriate authorization, a legitimate security purpose, and compliance with applicable privacy and data-protection requirements. Access to logs and predictions should be restricted to authorized personnel, and sensitive information should be protected through appropriate security controls. The system should not be used to discriminate against employees, make unsupported accusations, or automatically impose disciplinary actions. Human review, transparency, fairness, and accountability should guide the interpretation and use of all predictions. The project is intended to support responsible cybersecurity monitoring while respecting individual privacy and organizational policies.

2. Problem Statement:

Case Study scenario:

CyberShield Organization — Detecting Suspicious Employee Activity

CyberShield records every action performed by employees on its computer systems. A normal employee may repeatedly perform activities such as login, file access, browsing, and system operations. However, if an employee suddenly performs an unusual combination of activities, such as accessing a large number of files or generating abnormal system events, the security team needs to identify it quickly.

The organization has collected a large number of activity logs in the SPEDIA dataset. Manually examining these logs is time-consuming and makes it difficult to recognize unusual behavior.

The security team therefore wants to build a Machine Learning model in Python that learns the general pattern of employee activity and automatically flags events that significantly differ from the learned pattern.

A sample of 2,000 SPEDIA records is provided for analysis.

Objective:

Develop a Python-based Machine Learning system that detects anomalous user activity from SPEDIA logs and identifies the users/events that require further security investigation.

(a) Data Preparation

Using Python:

1. Load the SPEDIA dataset.
2. Select a random sample of 2,000 records.
3. Examine the dataset using head(), shape, info(), and describe().
4. Handle missing values and duplicate records.
5. Select relevant features such as:
   - User
   - Activity
   - Action
   - Decoder_name
   - Level
   - Size
   - Timestamp(c) Anomaly Detection

Use the Isolation Forest algorithm to detect unusual activity.

The model must:

1. Train using the prepared SPEDIA records.
2. Assign each event an anomaly prediction.
3. Separate events into:

Normal Activity

vs

Anomalous Activity

(b) Machine Learning Feature Engineering

Convert the selected log information into numerical features suitable for Machine Learning.
Perform:
- Categorical encoding
- Numerical feature processing
- Timestamp feature extraction where valid
- User/activity frequency features
Construct the final feature matrix

Assumptions

1. A suitable dataset containing user activity logs is available.
2. The dataset contains relevant features and reliable labels for classification.
3. Suspicious activities exhibit patterns that can be identified through machine learning.
4. Elastic Net Regularized Logistic Regression is suitable for classifying normal and potentially threatening activities.
5. Training data reasonably represents real-world user behavior.

Limitations

1. Model performance depends on dataset quality and representativeness.
2. False positives and false negatives may occur.
3. Novel threats and complex behavioural patterns may remain undetected.
4. Imbalanced datasets may reduce threat detection performance.
5. Changes in user behaviour may affect prediction accuracy.
6. Privacy concerns require secure data handling and authorized monitoring.
7. Predictions require human verification and do not establish malicious intent.

About the Dataset:

The dataset used in this project is the **SPEDIA Annotated Dataset** , designed to support research in insider-threat detection and cybersecurity analytics. It contains system and user activity logs that represent different types of events, including file operations, command execution, web activity, and email-related activities .The dataset includes features such as user and agent identifiers, timestamps, activity types, action types, file sizes, attachment information, and log levels. These attributes help the machine learning model identify patterns associated with normal and potentially malicious activities .For this project, a sample of **2,000 records** was selected from the annotated dataset to simplify data processing and model development. The target column, `Anomaly`, indicates whether an event is labelled as anomalous or non-anomalous:

- 0 – Non-anomalous activity

- 1 – Anomalous activity

The selected sample contains 850 non-anomalous records (42.5%) and 1,150 anomalous records (57.5%). Data preprocessing includes handling missing values, converting timestamps into useful time-based features, scaling numerical features, and encoding categorical features. Identifier and raw-text columns that are not used directly by the model

are excluded from the feature set.The prepared dataset is divided into training and testing sets using an 80:20 split. Stratified sampling is applied during the split to maintain the target-class proportions in both sets.

Purpose of EDA

Exploratory Data Analysis (EDA) was performed to understand the structure, characteristics, and quality of the SPEDIA Annotated Dataset before applying machine learning techniques. The main objective was to identify missing values, duplicate records, data distributions, relationships between features, and patterns associated with anomalous and non-anomalous activities.

EDA also helped determine suitable input features for the Elastic Net Regularized Logistic Regression model. Visualizations such as bar charts, histograms, pie charts, scatter plots, and a correlation heatmap were used to explore the dataset and understand the distribution of user activities and anomaly labels.

Preparation Steps Actually Performed

The following data preparation steps were carried out before model development:

1. **Dataset Loading:** The annotated SPEDIA dataset was loaded into a Pandas DataFrame using read_csv().
2. **Dataset Inspection:** The first five records, dataset dimensions, column names, data types, and statistical summary were examined.
3. **Missing Value Analysis:** Missing values in each column were identified using isnull().sum() to understand the completeness of the dataset.
4. **Duplicate Record Detection:** Duplicate rows were checked using duplicated().sum().
5. **Sampling:** A sample of 2,000 records was selected for manageable analysis and model development. The sample contained 850 non-anomalous records and 1,150 anomalous records.
6. **Target Variable Identification:** The Anomaly column was selected as the target variable, with 0 representing non-anomalous activity and 1 representing anomalous activity.
7. **Timestamp Processing:** The Timestamp column was converted into a datetime format. Additional time-based features, including hour, day, month, and day of the week, were extracted.
8. **Feature Selection:** Identifier and raw-text columns that were not directly used by the model were excluded from the input features.
9. **Data Visualization:** Bar charts, histograms, pie charts, scatter plots, and a correlation heatmap were created to examine feature distributions and relationships.
10. **Model Preparation:** The data was split into training and testing sets using an 80:20 stratified split. Numerical features were prepared using median imputation and

standardization, while categorical features were prepared using most-frequent-value imputation and One-Hot Encoding.

EDA Exploration – Implementation

EDA was implemented using Python libraries, primarily Pandas, NumPy, and Matplotlib. Pandas was used to load, inspect, summarize, and manipulate the dataset, while Matplotlib was used to generate visual representations of the data.

The exploration included the following activities:

- **Dataset Structure:** Examined the number of rows and columns, column names, and data types to understand the dataset's organization.
- **Statistical Analysis:** Used descriptive statistics to examine numerical features such as file size, attachment count, and log level.
- **Missing Values and Duplicates:** Checked for incomplete data and repeated records.
- **Target Distribution:** Used value counts and visualizations to examine the distribution of anomalous and non-anomalous events.
- **Categorical Feature Analysis:** Examined the distributions of activity types, action types, users, decoders, and agents using frequency counts and bar charts.
- **Numerical Feature Analysis:** Used histograms and scatter plots to investigate the distributions of numerical variables and relationships between selected features.
- **Time-Based Exploration:** Extracted temporal features from timestamps and examined event frequency by hour.
- **Correlation Analysis:** Generated a correlation matrix and heatmap to explore relationships among numerical features.
- **Feature and Target Relationships:** Used cross-tabulations and visualizations to compare anomaly labels across activity types, users, and decoder categories.

These steps provided an initial understanding of the data and helped prepare the features for the subsequent classification stage.

SPEDIA ANNOTATED DATASET — COMPACT EDA CODE

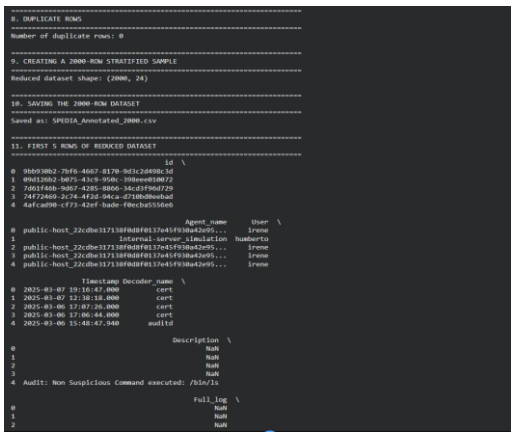
Essential inspection, sampling, visualizations, and final checks

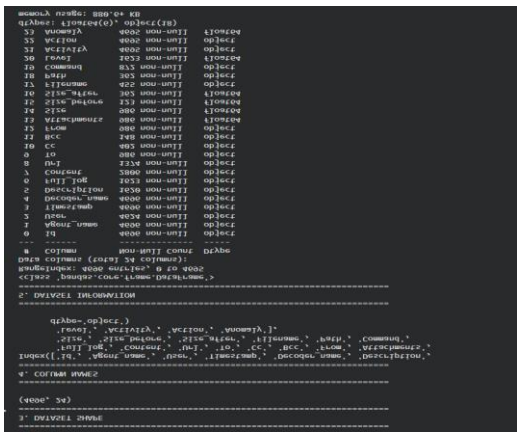
```
# SPEDIA ANNOTATED DATASET - COMPACT EDA
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
# 1. Load data and inspect structure
df = pd.read_csv("logs_SPEDIA_annotated_en.csv")
print("FIRST 5 ROWS")
print(df.head())
print("\nDATASET SHAPE:", df.shape)
print("\nCOLUMNS:", df.columns.tolist())
print("\nDATASET INFORMATION")
df.info()
print("\nSTATISTICAL SUMMARY")
print(df.describe(include="all").T)
print("\nMISSING VALUES")
print(df.isnull().sum())
print("\nDUPLICATE ROWS:", df.duplicated().sum())
 # 2. Create a 2,000-row sample while preserving the chosen class counts
normal = df[df["Anomaly"] == 0].sample(n=850, random_state=42)
anomaly = df[df["Anomaly"] == 1].sample(n=1150, random_state=42)
df = pd.concat([normal, anomaly], ignore_index=True)
df = df.sample(frac=1, random_state=42).reset_index(drop=True)
df.to_csv("SPEDIA_Annotated_2000.csv", index=False)
print("\nSAMPLED DATASET SHAPE:", df.shape)
print("\nANOMALY COUNTS")
print(df["Anomaly"].value_counts())
print("\nANOMALY PERCENTAGES")
print((df["Anomaly"].value_counts(normalize=True) * 100).round(2))
# 3. Important categorical summaries
```

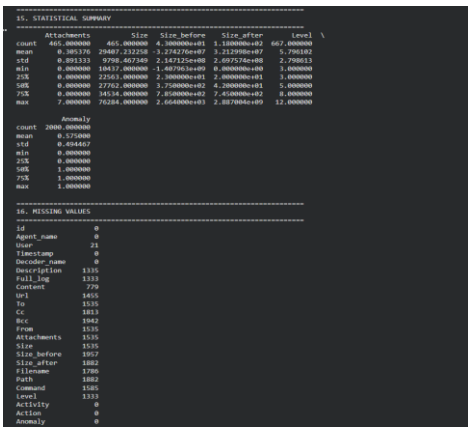
```
for col in ["Activity", "Action", "User", "Decoder_name", "Agent_name"]:
    if col in df.columns:
        print(f"\nVALUE COUNTS: {col}")
        print(df[col].value_counts().head(10))
# 4. Visualize anomaly distribution
df["Anomaly"].value_counts().sort_index().plot(kind="bar")
plt.title("Normal vs Anomalous Events")
plt.xlabel("Anomaly (0 = Normal, 1 = Anomaly)")
plt.ylabel("Number of Events")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
# 5. Visualize the most common activities
df["Activity"].value_counts().head(10).plot(kind="bar")
plt.title("Top 10 Activities")
plt.xlabel("Activity")
plt.ylabel("Number of Events")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
# 6. Numerical distributions
for col in ["Level", "Size", "Attachments"]:
    if col in df.columns:
        df[col].dropna().plot(kind="hist", bins=15, title=f"Distribution of {col}")
        plt.xlabel(col)
        plt.tight_layout()
        plt.show()
# 7. Timestamp analysis
df["Timestamp"] = pd.to_datetime(df["Timestamp"], errors="coerce")
df["Hour"] = df["Timestamp"].dt.hour
print("\nINVALID TIMESTAMPS:", df["Timestamp"].isna().sum())
print("\nEVENT COUNTS BY HOUR")
```

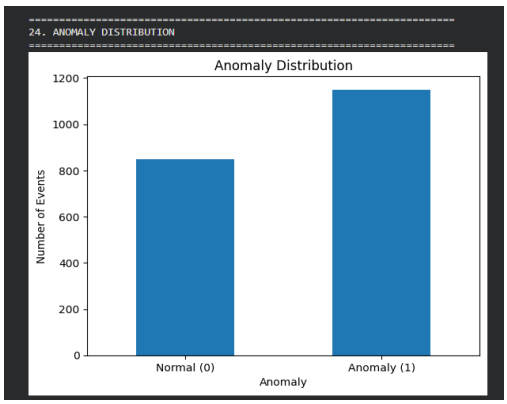
```
print(df["Hour"].value_counts().sort_index())
df["Hour"].value_counts().sort_index().plot(kind="bar")
plt.title("Events by Hour")
plt.xlabel("Hour of Day")
plt.ylabel("Number of Events")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
# 8. Correlation heatmap for numerical columns
numeric_cols = df.select_dtypes(include="number").columns
correlation = df[numeric_cols].corr()
plt.figure(figsize=(9, 6))
sns.heatmap(correlation, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()
# 9. Compare activity type with anomaly label
activity_anomaly = pd.crosstab(df["Activity"], df["Anomaly"])
print("\nACTIVITY VS ANOMALY")
print(activity_anomaly)
activity_anomaly.plot(kind="bar")
plt.title("Activity vs Anomaly")
plt.xlabel("Activity")
plt.ylabel("Number of Events")
plt.xticks(rotation=45, ha="right")
plt.legend(title="Anomaly")
plt.tight_layout()
plt.show()
# 10. Compare event size with attachment count
scatter = df[["Size", "Attachments"]].dropna()
plt.scatter(scatter["Size"], scatter["Attachments"], alpha=0.6)
plt.title("Event Size vs Attachments")
```

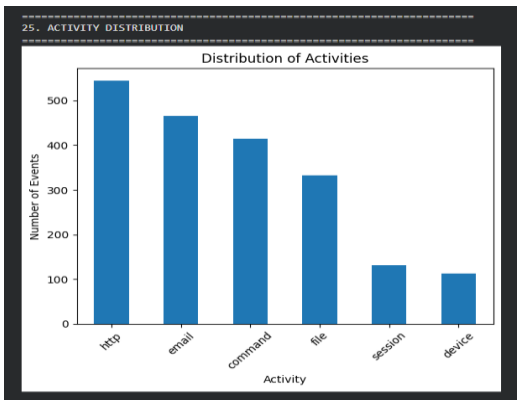
```
plt.xlabel("Event Size")
plt.ylabel("Number of Attachments")
plt.tight_layout()
plt.show()
# 11. Final checks
print("\nFINAL DATASET SHAPE:", df.shape)
print("\nFINAL MISSING VALUES")
print(df.isnull().sum())
print("\nFINAL DUPLICATE ROWS:", df.duplicated().sum())
print("\nFINAL ANOMALY DISTRIBUTION")
print(df["Anomaly"].value_counts()) print("\nEDA COMPLETED")
```

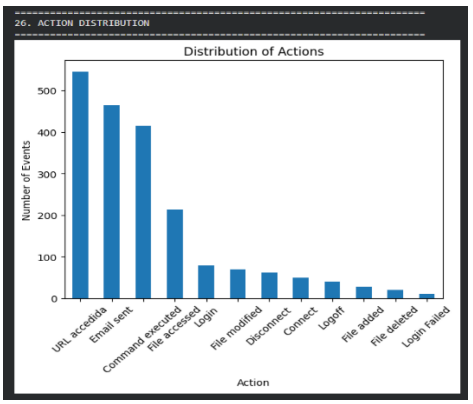


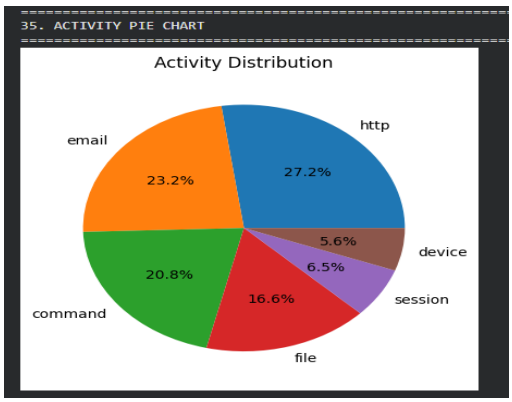


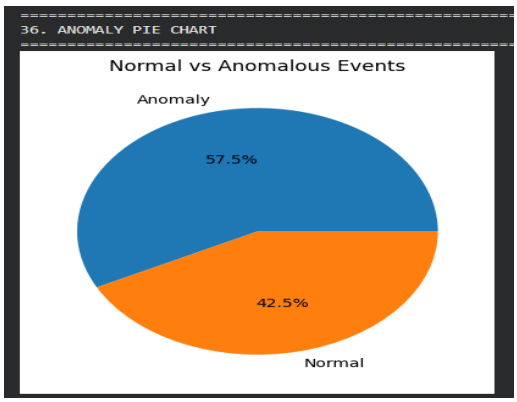


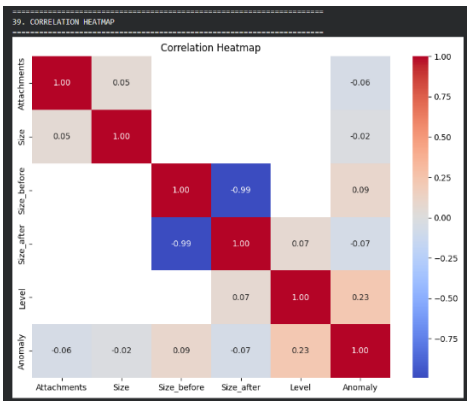


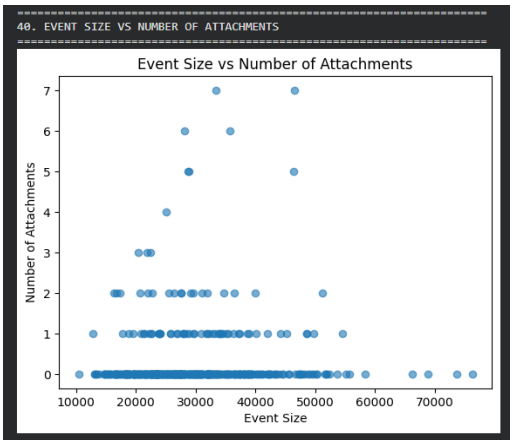


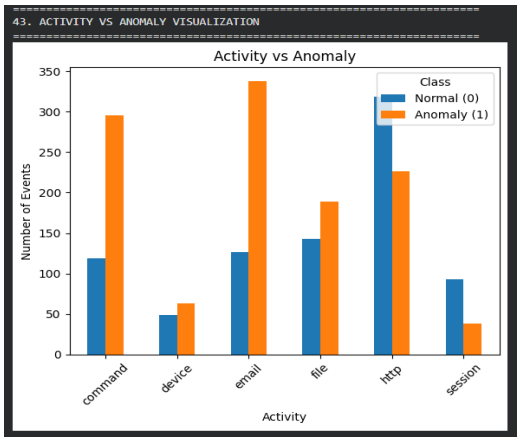












2. Algorithm Selection

The algorithm selected for this project is **Elastic Net Regularized Logistic Regression**. Logistic Regression is a supervised machine learning algorithm commonly used for binary classification. In this project, it learns patterns from the annotated activity logs to estimate the probability that an event is anomalous. Elastic Net combines two regularization techniques: **L1 (Lasso) regularization** and **L2 (Ridge) regularization**. L1 regularization can encourage less useful feature coefficients to become zero, while L2 regularization helps reduce excessively large coefficients. Combining both techniques helps control model complexity and can improve generalization to unseen data. The model is implemented using the `LogisticRegression` class from Scikit-learn with the following configuration:

- **Penalty:** Elastic Net

- **Solver:** SAGA

- **L1 ratio:** 0.5, giving a balanced combination of L1 and L2 regularization

- **Inverse regularization strength (C):** 1.0

- **Maximum iterations:** 5,000

Before training, numerical features are processed using median imputation and standardization. Categorical features are processed using most-frequent-value imputation and One-Hot Encoding. These preprocessing steps are combined with the classifier in a Scikit-learn pipeline to ensure that the same transformations are applied during training and prediction. The model is evaluated using accuracy, precision, recall, F1-score, ROC-AUC, and a confusion matrix. These metrics provide different perspectives on classification performance, particularly because incorrectly identifying a malicious event as safe may be a significant concern in cybersecurity applications. In addition to predicting the probability of an anomalous event, the project maps that probability to three user-friendly risk categories:

- **SAFE:** Probability below 0.34

- **PARTIAL THREAT:** Probability from 0.34 to below 0.67

- **THREAT:** Probability of 0.67 or higher

These categories are derived from the model's predicted probability; the model itself is trained on the dataset's original binary `Anomaly` labels.

Elastic Net Regression : Compact Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, roc_auc_score

df = pd.read_csv("SPEDIA_Annotated_2000.csv")

df["Timestamp"] = pd.to_datetime(df["Timestamp"], errors="coerce")
for col, values in {
    "Hour": df["Timestamp"].dt.hour,
    "Day": df["Timestamp"].dt.day,
    "Month": df["Timestamp"].dt.month,
    "DayOfWeek": df["Timestamp"].dt.dayofweek
}.items():
    df[col] = values

y = df["Anomaly"]
X = df.drop(columns=[
    "Anomaly", "id", "Timestamp", "Description",
    "Full_log", "Content", "Url", "To", "Cc", "Bcc",
    "From", "Filename", "Path", "Command"
], errors="ignore")

num = X.select_dtypes(include="number").columns
cat = X.select_dtypes(include=["object", "category"]).columns

preprocessor = ColumnTransformer([
    ("num", Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scale", StandardScaler())
    ]), num),
    ("cat", Pipeline([
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("encode", OneHotEncoder(handle_unknown="ignore"))
    ]), cat)
])

model = Pipeline([
    ("prep", preprocessor),
    ("clf", LogisticRegression(
        penalty="elasticnet", solver="saga",
        l1_ratio=0.5, C=1.0, max_iter=5000
    ))
])
```

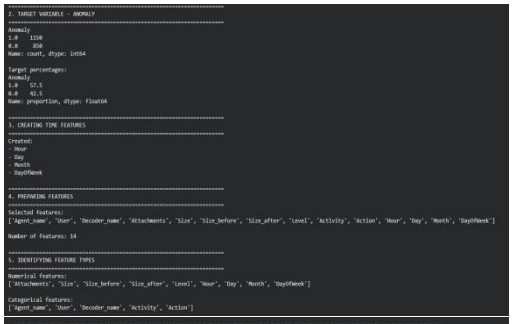
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

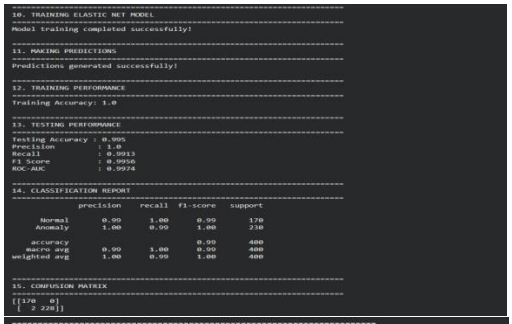
model.fit(X_train, y_train)
pred = model.predict(X_test)
prob = model.predict_proba(X_test)[:, 1]

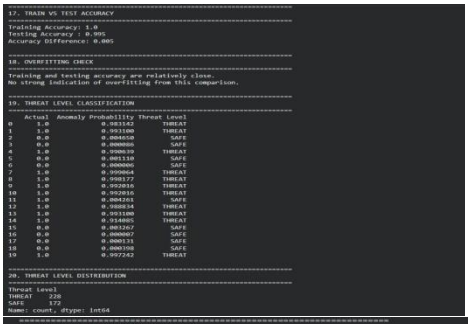
print("Accuracy:", accuracy_score(y_test, pred))
print(classification_report(y_test, pred, zero_division=0))
print("ROC-AUC:", roc_auc_score(y_test, prob))

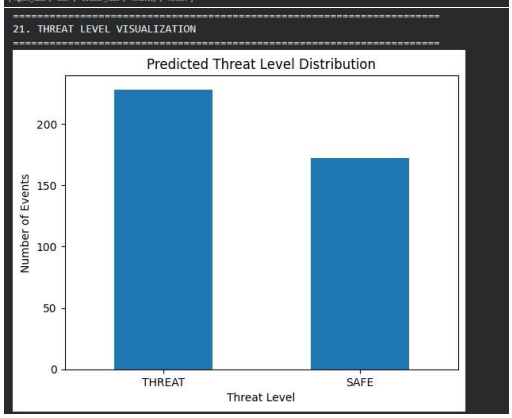
def threat(p):
    return "THREAT" if p >= 0.67 else "PARTIAL THREAT" if p >= 0.34 else "SAFE"

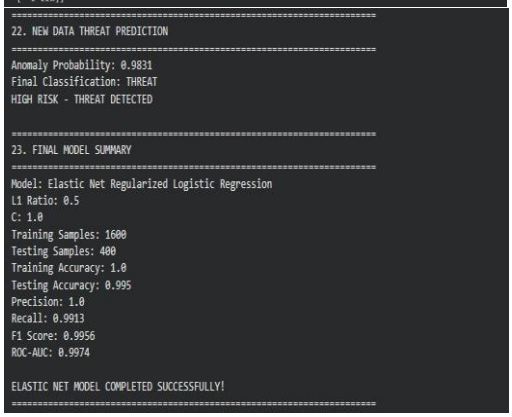
print("Threat Level:", threat(prob[0]))
```

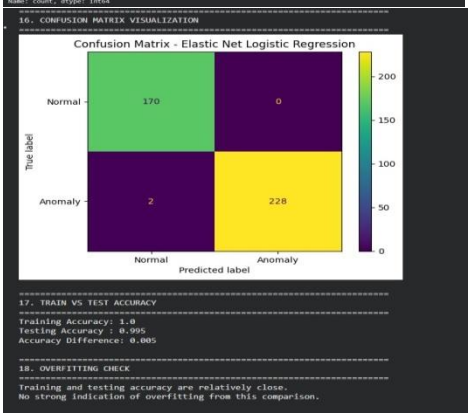












3. Conclusion

This project demonstrates how machine learning can be applied to cybersecurity log analysis to identify potentially anomalous user and system activities. The SPEDIA Annotated Dataset provides labelled activity records that allow the model to learn patterns associated with anomalous and non-anomalous events. Elastic Net Regularized Logistic Regression was selected because it combines the interpretability of Logistic Regression with L1 and L2 regularization. The preprocessing pipeline handles missing values, scales numerical features, and converts categorical data into a format suitable for model training. The model's performance is assessed using multiple evaluation metrics, including accuracy, precision, recall, F1-score, and ROC-AUC. The predicted anomaly probabilities are also converted into three risk categories—SAFE, PARTIAL THREAT, and THREAT—to make the results easier to interpret. Overall, the project provides a structured baseline for machine-learning-based insider-threat detection. However, an anomalous event should not automatically be treated as confirmed malicious activity. The risk categories are probability-based indicators and should be interpreted alongside additional evidence. Further improvements could include hyperparameter tuning, cross-validation, testing on larger datasets, and evaluating the model on data collected from a different time period or environment.